

DATA & MACHINE LEARNING CLUB

Git & GitHub Desktop Guide

Version Control • Course Materials • Collaboration
LUANAR, Lilongwe | 2026

This guide covers everything a DML Club member needs to know about Git and GitHub Desktop — from creating an account to collaborating on club projects. All lecture notes, session handouts, code samples, and project solutions are distributed through the club GitHub repository, so every member should work through this guide carefully.

01 WHAT IS GIT AND GITHUB?

Git is a version control system, it tracks every change made to files over time, so you can always go back to an earlier version, see exactly what changed, and collaborate with others without overwriting each other's work. Think of it like a detailed save-history for your entire project folder.

GitHub is a website that hosts Git repositories (project folders) in the cloud. It makes it easy to share files with teammates, download course materials, and contribute to projects from anywhere in the world, all for free.

Key Concepts — Glossary

Term	Plain-English Meaning
Repository (Repo)	A project folder tracked by Git — stores all files and their complete change history
Commit	A saved snapshot of your changes, with a short message describing what you did
Push	Upload your local commits to GitHub so others can access them
Pull / Fetch	Download the latest changes from GitHub onto your local computer
Clone	Download a complete copy of a GitHub repository to your computer for the first time
Branch	A separate line of work — experiment freely without affecting the main project
Merge	Combine changes from one branch back into another (usually into main)
Main / Master	The primary branch — the official, stable version of all project files

README.md	A text file shown on the repo homepage — describes the project and contains links
.gitignore	Lists files Git should ignore and NOT track (e.g. large data files, system files)
Origin	The name GitHub Desktop uses to refer to the remote copy of your repo on GitHub
Conflict	When two people change the same line of a file differently — Git asks you to decide which to keep

02 CREATING YOUR GITHUB ACCOUNT

1. Go to github.com in your web browser
2. Click 'Sign up' in the top-right corner
3. Enter your email address, choose a username, and create a password
4. Verify your email address when GitHub sends a confirmation message
5. Your public profile is now live at github.com/your-username

🔴 **CHOOSE A PROFESSIONAL USERNAME** - it is permanent and visible to everyone. Recruiters, universities, and future collaborators will see it.
Use your real name or initials: e.g. hobby-bwanali or marting-chirwa.
Avoid casual nicknames, numbers, or anything you might regret professionally.

03 INSTALLING GITHUB DESKTOP

GitHub Desktop is the recommended tool for all DML Club members. It lets you do everything Git-related through a visual interface, no typing commands required.

6. Go to desktop.github.com and download GitHub Desktop for your operating system (Windows or macOS)
7. Run the installer — GitHub Desktop opens automatically when the installation is complete
8. On the welcome screen, click 'Sign in to GitHub.com'
9. Enter your GitHub username and password, then click 'Authorize desktop'
10. Set your display name and email address — these will appear on all your commits
11. Click 'Finish' — GitHub Desktop is now connected to your account and ready to use

04 CLONING THE DML CLUB REPOSITORY

Cloning is how you download the club repository to your computer for the first time. Once cloned, you have all course materials locally — lecture notes, handouts, code samples, and solutions — and you can update them with one click whenever new content is uploaded.

STEP 1 Open GitHub Desktop and go to File > Clone repository...

STEP 2 Click the 'URL' tab at the top of the dialog box

STEP 3 Paste the DML Club repository URL (shared by facilitators each session)

STEP 4 Under 'Local path', choose where to save files — e.g. Documents/DML-Club

STEP 5 Click 'Clone' — GitHub Desktop downloads everything to your chosen folder

STEP 6 Click 'Open in Explorer / Finder' to browse all the downloaded files

📌 DML CLUB REPOSITORY FOLDER STRUCTURE:

/Lectures — Session slide decks and lecture notes (PDF)
/References_materials — Reference sheets and quick guides (including this one)
/Projects — Club projects – in progress or completed

05 GETTING UPDATES — PULLING NEW MATERIALS

After the initial clone, you never need to clone again. Whenever there are new lecture notes, handouts, or solutions, you simply 'pull' those changes to your local copy. This takes about 10 seconds.

STEP 1 Open GitHub Desktop

STEP 2 Select the DML Club repository from the left panel

**STEP
3**

Click 'Fetch origin' at the top — checks if any new changes exist online

**STEP
4**

If 'Pull origin (X commits)' appears, click it to download the new files

**STEP
5**

Open your local folder — new lectures, handouts, or solutions are now there



START OF SESSION HABIT: In every DML Club session, open GitHub Desktop and pull the latest changes. This ensures you always have the most current handouts and code samples to work with. Make it the first thing you do every single session.

06 CREATING YOUR FIRST REPOSITORY

Every member should create a personal repository to store their own practice work — code written during sessions, personal experiments, and project submissions. This becomes your portfolio of skills built during the club.

12. Open GitHub Desktop and click 'File > New repository...'
13. Enter a name — no spaces, use hyphens (e.g. dml-practice or pico-projects)
14. Add a short description such as: 'My DML Club practice projects — LUANAR 2026'
15. Set the Local path to where you want the folder created on your computer
16. Tick 'Initialize this repository with a README' — this creates the first file automatically
17. Leave 'Git ignore' as None for now (you can configure it later)
18. Click 'Create repository' — the repo folder is created on your computer
19. Click 'Publish repository' to upload it to GitHub
20. Choose Private (only you) or Public (visible to anyone) and confirm



YOUR GITHUB PROFILE becomes a living portfolio of everything you build. Public repositories are visible to anyone — including future employers. As you complete club projects and upload your work, you are actively building a record of your data science and hardware skills.

07 SAVING YOUR WORK — COMMIT & PUSH

When you create or edit files inside your repository folder, Git automatically detects the changes. To officially save those changes and share them on GitHub, you perform two steps: Commit (create a local snapshot) then Push (upload to GitHub).

STEP 1 Save your file(s) inside the repository folder on your computer

STEP 2 Open GitHub Desktop — changed files appear automatically in the middle panel

STEP 3 Review the changes: green lines = additions, red lines = deletions

STEP 4 In the bottom-left box, type a short Summary message describing what you did

STEP 5 Click the blue 'Commit to main' button — snapshot saved locally

STEP 6 Click 'Push origin' at the top — your work is now live on GitHub

Writing Good Commit Messages

A commit message is a short note describing what you changed. Good messages make your history readable — for yourself later, and for collaborators reviewing your work.

Good Commit Messages	Why They Work
Add Project 1 LED blink code	Specific — names the project and what was added
Fix DHT11 timing delay in Project 5	Describes exactly what was fixed and where
Add notes from Session 2 handout	Clear content description, easy to find later
Rename variables for clarity in traffic light	Explains the purpose of the change
Update README with session schedule	Names the file and what changed in it

⚠️ **AVOID VAGUE MESSAGES:** 'changes', 'stuff', 'update', 'asdfgh', 'final v2', 'idk', or 'fix'. These make it impossible to find specific changes later. If you cannot describe your change in one line, break it into smaller commits.

08 VIEWING HISTORY & UNDOING CHANGES

One of Git's greatest strengths is that nothing is ever permanently lost. You can review every change ever made and restore any previous version of any file.

Viewing History

- In GitHub Desktop, click the 'History' tab in the left panel
- Every commit is listed in order, newest at the top — click any one to inspect it
- The right panel shows exactly which lines were added (green) or removed (red) in each file
- You can also see the full history of any repository on github.com under the 'Commits' link

Undoing & Recovering

Situation	How to Fix It
I committed but have not pushed yet	History tab > right-click the commit > 'Undo commit'
I made changes but have not committed	Changes panel > right-click the file > 'Discard changes'
I want to restore a file to an older version	History tab > click the commit > right-click the file > 'Checkout file'
I pushed a mistake to GitHub	Make a new corrected commit and push again — never force-delete history
I accidentally deleted a file	History tab > find the last commit that had it > 'Checkout file'



Think of commits exactly like save points in a video game — you can always reload an earlier save if something breaks. This is why we commit frequently and always commit BEFORE trying something new or experimental. A good habit: commit at the end of every working session, even small ones.

09 BRANCHES — EXPERIMENTING SAFELY

A branch is an independent copy of your project where you can try new ideas, write experimental code, or make big changes — without touching the main working version. When the experiment succeeds, you merge it back. If it fails, simply delete the branch.

Creating and Using a Branch

21. In GitHub Desktop, click the 'Current Branch' button at the top of the window
22. Click 'New branch' and give it a descriptive name (e.g. try-pwm-fade, sensor-test, add-logging)
23. Click 'Create branch' — you are now on the new branch; main is untouched
24. Make changes and commit them as normal — all commits go to this branch only

25. To switch back to main, click 'Current Branch' and select 'main'
26. When satisfied with the experiment, click 'Current Branch > Choose a branch to merge...'
27. Select your experiment branch and click 'Merge into main'
28. If the branch is no longer needed, you can delete it from the branch list

✦ **FOR CLUB PROJECT SUBMISSIONS:** You may submit work on a branch named after yourself (e.g. mchirwa-project2 or hobby-submission). This keeps everyone's work separate and makes reviewing submissions easy. Always check session instructions for the exact branch naming convention.

10 USING GITHUB ON THE WEB

GitHub.com is always available from any browser — no software needed. This is useful for quickly reading files, downloading specific materials, or checking the club repository from a phone or shared computer.

Task	How to Do It on github.com
Browse repository files	Go to github.com/username/repo-name and click any folder or file
Download a single file	Open the file, click the download icon at the top-right of the file view
Download everything as a ZIP	On repo homepage, click the green 'Code' button > 'Download ZIP'
Read the README	It appears automatically on the repository homepage — check it for session links
View commit history	Click the clock icon or 'X commits' link on the repo homepage
Report an issue / ask a question	Click the 'Issues' tab > 'New issue' > describe your question or problem
See another member's repo	Go to github.com/their-username and browse their public repositories

11 QUICK REFERENCE — ALL COMMON ACTIONS

Action	How to Do It in GitHub Desktop
Clone a repo (first download)	File > Clone repository > URL tab > paste URL > choose path > Clone
Get latest updates from GitHub	Select repo > Fetch origin > Pull origin (if updates available)
Save changes as a local snapshot	Changes panel > write Summary message > Commit to main
Upload commits to GitHub	Click 'Push origin' after committing
Create a new repository	File > New repository > fill in details > Create repository > Publish
Create a new branch	Current Branch button > New branch > name it > Create branch
Switch to a different branch	Current Branch button > click the desired branch name
View all past commits	Click the 'History' tab in the left panel
Undo last commit (not pushed)	History tab > right-click the commit > Undo commit
Discard file changes (not committed)	Changes panel > right-click file > Discard changes
Restore a file from history	History tab > right-click file in that commit > Checkout file
Merge a branch into main	Current Branch > Choose a branch to merge into main > select branch > Merge

12 THE DML CLUB WORKFLOW — SESSION BY SESSION

When	What to Do
First time only	Create GitHub account > Install GitHub Desktop > Clone DML Club repository
Start of every session	Open GitHub Desktop > Fetch origin > Pull origin to get latest materials
During a session	Write your code in the repository folder, save files normally
End of every session	Stage changes in GitHub Desktop > write commit message > Commit > Push
Submitting a project	Create a branch named as instructed > commit your work there > Push > notify facilitator
Something goes wrong	Use History tab to find last good state > Checkout file or Undo commit

13 USEFUL LINKS & RESOURCES

- GitHub: github.com
- GitHub Desktop: desktop.github.com

- GitHub Docs (official help): docs.github.com
- GitHub Student Developer Pack (free tools for students): education.github.com/pack
- DML Club Repository: see facilitator for the latest URL



◆ DML CLUB CONTACTS

Hobby Bwanali: hobbybwanali@gmail.com | +265 885 134 501

Martin Chirwa: martinchirwa508@gmail.com | +265 997 253 881

Mentor: Dexter Shepherd — University of Sussex, ISAID Team

Location: LUANAR, Lilongwe, Malawi

From Data to Decisions — Building Tomorrow's Innovators Today

Data & Machine Learning Club © 2026